

Group_129,_Sentiment_Analysis_in_Social_Media_Part_B

August 19, 2025

1 Group Information

- **Group No:** 129
- **Members:**
 - Elluru Sai Gagan – 2024AB05190
 - Amulya Gupta – 2024AB05200
 - Akash Shivadas Chatake – 2024AB05226
 - Kintul Upadhyay – 2024AB05261

2 Journal Used for Implementation

- **Title:** *Sentiment Analysis of IMDb Movie Reviews Using Long Short-Term Memory*
- **Authors:** Muhammad Qaisar, Ammara Nosheen, Hafeez Ur Rehman Siddiqui
- **Venue/Journal, Year:** *International Journal of Advanced Computer Science and Applications (IJACSA)*, 2020
- **Citation (APA 7th):**

Qaisar, M., Nosheen, A., & Siddiqui, H. U. R. (2020). Sentiment analysis of IMDb movie reviews using long short-term memory. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 11(5), 386–394. <https://doi.org/10.14569/IJACSA.2020.0110549>
- **Link/DOI:** <https://doi.org/10.14569/IJACSA.2020.0110549>

3 Library Imports

```
[ ]: import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras import losses
from tensorflow.keras import models
import keras
from keras import regularizers
```

```
[ ]: import os
import re
import shutil
import string
import numpy as np
import seaborn as sns
import pickle
```

```
[ ]: from sklearn.metrics import classification_report, accuracy_score
from sklearn.metrics import confusion_matrix, RocCurveDisplay
from sklearn.metrics import roc_curve, auc
```

```
[ ]: import matplotlib.pyplot as plt
%matplotlib inline
```

4 Acquiring the Dataset

```
[ ]: # accessing the imdb dataset
url = "https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz"

dataset = tf.keras.utils.get_file("aclImdb_v1", url,
                                   untar=True, cache_dir='.',
                                   cache_subdir='')

datasetDir = os.path.join(os.path.dirname(dataset), 'aclImdb_v1')
```

Downloading data from
https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz
84125825/84125825 2s
0us/step

```
[ ]: # list of contents of the dataset main directory
os.listdir(datasetDir)
```

```
[ ]: ['aclImdb']
```

```
[ ]: # exploring the training section
trainDir = os.path.join(datasetDir, 'aclImdb/train')
os.listdir(trainDir)
```

```
[ ]: ['unsup',
      'urls_pos.txt',
      'labeledBow.feats',
      'pos',
      'urls_unsup.txt',
      'neg',
```

```
'urls_neg.txt',  
'unsupBow.feats']
```

```
[ ]: # positive reviews  
os.listdir(os.path.join(trainDir, "pos"))[:10]
```

```
[ ]: ['4162_10.txt',  
      '8076_7.txt',  
      '3822_10.txt',  
      '11517_9.txt',  
      '432_8.txt',  
      '10353_9.txt',  
      '699_8.txt',  
      '6517_10.txt',  
      '231_10.txt',  
      '7925_7.txt']
```

```
[ ]: # opening a sample negative review  
with open(os.path.join(os.path.join(trainDir, "neg"),  
                        os.listdir(os.path.join(trainDir, "neg"))[80])) as f:  
    print(f.read())
```

Basically this is about a couple who want to adopt a second child. At the adoption agency they meet a mouse (Stuart) and they decide to adopt him. If you think that this is stupid, hold it, because it's getting worse.

Stuart arrives to his new home, where he is treated like a human child. (Spare me!) The rest is pretty much the usual cliché, about family problems, jealousy from the elder "brother", and at the end all issues are resolved and they are all a "happy family". Boring and worn out as this is, it is also shown in the most blunt and unsophisticated way.

I don't know if the director believed that he was being creative by introducing a mouse to the cliché, or he was just trying to fill in minutes, but he only upgraded the cliché from boring to abhorrent.

Then why I gave a 3 and not a ZERO? Because of the family cat, who loves Stuart as much as the "brother". And because of some funny gigs, where Stuart makes good use of his small size.

On the overall I believe that the film would work reasonably well if: a. Stuart was a PET and not a "sibling". b. It had kept to the funny gigs, like Stuart trying to outwit the cat, and had left out boring clichés which don't even match with anything else.

```
[ ]: # removing the unsup subdirectory  
shutil.rmtree(os.path.join(trainDir, "unsup"))  
os.listdir(trainDir)
```

```
[ ]: ['urls_pos.txt',  
      'labeledBow.feats',  
      'pos',  
      'urls_unsup.txt',
```

```
'neg',  
'urls_neg.txt',  
'unsupBow.feats']
```

5 Data Partitioning

```
[ ]: # the dataset already has two partitions  
# in the test and train subdirectories  
# train will be partitioned into train and validation  
  
# partition ratio - validation as a fraction of train whole  
PARTITION_VAL = 0.2
```

```
[ ]: BATCH_SIZE = 128  
SEED = 42
```

```
[ ]: # train partition 1 without preprocessing  
rawTrain1 = tf.keras.utils.text_dataset_from_directory(  
    os.path.join(datasetDir, 'aclImdb/train'),  
    batch_size = BATCH_SIZE,  
    validation_split = PARTITION_VAL,  
    subset = 'training',  
    seed = SEED  
)
```

Found 25000 files belonging to 2 classes.
Using 20000 files for training.

```
[ ]: # few samples of the training partition  
for textBatch, labelBatch in rawTrain1.take(1):  
    for i in range(20, 30):  
        print("Review: ", textBatch.numpy()[i])  
        print("Label: ", labelBatch.numpy()[i])  
print("Where")  
print("Label 0 =", rawTrain1.class_names[0])  
print("Label 1 =", rawTrain1.class_names[1])
```

Review: b"When I heard the plot for this movie I simply had to see it, I mean whole cities being wiped out by killer tomatoes! Sadly the title is about as funny as it gets.

Led by Detective Dick Mason, a special team of military and scientists (including Greg Colburn who never takes his SCUBA outfit off and Lt. Finletter who is never pictured without his parachute trailing behind) 'Attack Of The Killer Tomatoes' is a parody of B-Movies, in particular Japanese horror of the 1950's. The film begins with a standard sized tomato being discovered by a women washing up in her kitchen before we find ourselves in a middle of a crime scene as the tomato has supposedly murdered this lady,

and let me tell you it doesn't get any saner as the film progresses! To be fair there are a couple of funny moments, for instance anytime the Japanese scientist Dr Nokitofa speaks his voice is dubbed over in an American accent, or when disguise expert Sam Smith infiltrates the tomatoes 'hey, can somebody please pass the ketchup?'. Equally this film was probably a lot funnier in 1978 with the whole so bad its good concept. Unfortunately for 'Attack Of The Killer Tomatoes' spoof films such as the 'Airplane' and 'Naked Gun' series have been released and done this kind of comedy a lot better since.

The acting is atrocious; there is zero continuity in the editing and it just feels genuinely slow and lacking energy. For a parody film to work you need a lot of things happening at once, one gag after the over. The singing in the film seems pointless and the adverts for the furniture store that flash across the screen are damn right bizarre, even for this film. Ultimately, however, you can see why this film is a cult one; I can't see many people being indifferent to it. Unfortunately terrible would be the way I would sum this up."

Label: 0

Review: b'To compare this squalor with an old, low budget porno flick would be an insult to the old, low budget porno flick. The animal scenes have no meaning nor do they represent this man and his crimes even in the broadest sense of abstractions. The synopsis on the back of the DVD case says in part, "\xc2\x85gripping retelling of the BTK Killer\'s reign of terror." This is NOT a retelling. A retelling would suggest that you are being told the truth of what happened or how or why. None of these things are true. I\'m an enthusiastic studier of serial killers and have seen some pretty crappy movies about them and honestly, this IS NOT one of them. This isn\'t even about the BTK killer. Save yourself some time and a few bucks and rent Dahmer instead. THAT serial killer movie is accurate and true. However, if you just HAVE to see this movie for yourself, check it out for free at your local library and even then, you\'ll still feel cheated.'

Label: 0

Review: b"The dancing was probably the ONLY watchable thing about this film -- and even that was disappointing compared to some other films. My gawd!

To me, this is the worst kind of film -- one that assumes it's a work of art because it has all the trappings of film-as-art. Yes, it's beautifully photographed, but ultimately lacks the depth and tension of the dance around which the film supposedly surrounds itself. Tango is a tease, it's hot, it has drama, it's audacious -- precisely what this film is not."

Label: 0

Review: b"The sequel to the ever popular Cinderella story reminded me somewhat of what they did with one of the Beauty & the Beast movies. It's basically three short stories rolled into 1.

OK, the mice are adorable (I love Gus! He's sooo cute!), and Lucifer's awesome (as usual). I liked some of the newer characters as well, (Pom Pom was adorable and I did like Prudence.). Still, the storyline was somewhat limited, but still very cute. So, I vote 7/10."

Label: 1

Review: b'From the very opening scene you will notice just how hard they tried to mimic the very smart and powerful \'Cruel Intentions\', and how flat it landed. You\'ll also notice what a terrible choice they made by casting Robin

Dunne as Valmont... Then in the second scene, you meet the two best things in this movie, Amy Adams and Mimi Rogers as Kathryn and her mother. That is, if you can get past the fact that Kathryn wasn't blonde in the first film... Then the movie goes on, you see the cheap romantic story from miles ago, and you notice Sebastian has already met an Anette in the past, here called Danielle, and a Cecile, here called Cherie... How original is that for a prequel. Then it turns into a low budget \"Wild Things\" type of film with lots and lots of oh-my \"twists\". As I mentioned, Robin Dunne was a very bad choice. Not that he is a bad actor, he's good.. He just doesn't have the charisma Ryan did. Amy Adams, who is in my opinion one of the most talented young actresses of our time, once again delivers. But with all the talent in the world, there is no way one could save this trash. As a whole, this \"movie\" feels like a \"Beverly Hills, 90210\" episode. The score has been stolen from \"Cruel Intentions\" and \"Jawbreaker\"... Yes, they used the score from JAWBREAKER... Couldn't they at least leave that one alone?! You'll want to pass this one. If you want more Cruel Intentions, watch Stephen Frears' Dangerous Liaisons.'

Label: 0

Review: b\"One of the most frightening game experiences ever that will make you keep the lights on next to your bed. Great storyline with a romantic, horrific, and ironic plot. Fans of the original Resident Evil will be in for a surprise of a returning character! Not to mention that the voice-acting have drastically improved over the previous of the series. Don't miss out on the best of the series.\"

Label: 1

Review: b\"Live Feed is set in some unnamed Chinese/Japanese Asian district somewhere as five American friends, Sarah (Ashley Schappert), Emily (Taayla Markell), Linda (Caroline Chojnacki), Mike (Lee Tichon) & Darren (Rob Scattergood) are enjoying a night on the town & taking in the sights. After a scuffle in a bar with a Japanese Triad boss (Stephen Chang) they decide to check out a porno theatre, as you would. Inside they are separated & quickly find out that the place belongs to the Triad boss who uses it to torture & kill people for reasons which aren't made clear. Can local boy Miles (Kevan Ohtsji) save them?

This Canadian production was co-written, produced & directed by Ryan Nicholson who also gets a prosthetic effects designer credit as well, one has to say that Live Feed is another pretty poor low budget shot on a camcorder type horror film that seems to exist only to cash in on the notoriety & success of Hostel (2005) & the mini craze for 'torture porn' as it's become known. According the IMDb's 'Trivia' section for Live Feed writer & director Nicholson wrote it after hearing about certain activities taking place in live sex theatres, for my money I reckon he wrote it after watching Hostel! The script is pretty poor, there is no basic reason given as to why this porno theatre has a big fat ugly freak dressed in bondage gear lurking around torturing & killing people, none. Was it for the Triads? Was it for his pleasure? Was it to make snuff films to sell? Some sort of explanation would have been nice. Also why did he turn on the Triad boss at the end? If your looking for a film with a coherent story then forget about Live Feed. It seemed to me to be some sort of uneasy misjudged mix of sex, S&M, horror, torture, gore & action films which doesn't come off. I mean just setting a horror film in a porn theatre isn't

automatically going to make your film any good, there still needs to be a decent script & story, right? The character's were fairly poor cliché & some of their actions & motivations were more than a little bit questionable. It moves along at a reasonable pace, it's fairly sleazy mixing gore, sex & nudity but it does look cheap which lessens the effect.

Director Nicholson doesn't do anything special here, the editing is choppy & annoying, he seems to think lighting almost every scene with neon lights is a good idea & the film has a cheap look about it. Available in both 'R' & 'Unrated' versions I saw the shorter cut 'R' version which really isn't that gory but I am prepared to give the benefit of the doubt to the 'Unrated' version & say that it might be much, much gorier but I can't say for sure. There's a fair amount of nudity too if that's your thing. I wouldn't say there's much of an atmosphere or many scares here because there isn't & aren't respectively although it does have a sleazy tone in general which is something it has going for it I suppose.

Technically Live Feed isn't terribly impressive, the blood looks a little too watery for my liking & entire scenes bathed in annoying neon lights sometimes makes it hard to tell what's happening, it too often looks like it was shot on a hand-held camcorder & the choppy editing at least on the 'R' rated version is at times an annoying mess. Shot on location in an actual porn theatre somewhere in Vancouver in Canada. The acting is poor, sometimes I couldn't tell if the actresses in this were supposed to be crying or laughing...

Live Feed is not a film I would recommend anyone to rush out & buy or rent, I didn't think much of it with it's very weak predictable storyline lacking exposition & which goes nowhere, poor acting & less than impressive gore (at least in the 'R' rated cut anyway). Watch either Hostel films again or instead as they are superior."

Label: 0

Review: b"A lot of the problem many people have with this movie is that they seem to think that the story should have been more entertaining (ignoring it is based on a true story) or ranting against a film that glorifies Che (which it really doesn't). This film is very close to Jon Anderson's definitive bio on Che and gets the story right. Soderburgh does an excellent job of setting the mood for the unraveling debacle that was Che's Bolivian adventure. You really get the impression of the total timidity and bewilderment of the Bolivian peasant to Che's revolutionary ideas or of the difficulties that his men faced with hunger and the terrain. Sorry to bore the attention challenged movie fan out there but that was how it happened. So don't go into this movie expecting a Rambo shoot em up, it's a true story!"

Label: 1

Review: b"Highly enjoyable, very imaginative, and filmic fairytale all rolled into one, Stardust tells the story of a young man living outside a fantasy world going inside it to retrieve a bit of a fallen star only to find the star is alive, young, and beautiful. A kingdom whose king is about to die has said king unleash a competition on his several sons to see who can retrieve a ruby first to be king whilst a trio of witches want the star to carve up and use to keep them young. These three plot threads weave intricately together throughout the entire picture blended with good acting, dazzling special effects, and some solid sentiment and humour as well. Stardust is a fun film and has some fun performances from the likes of Claire Danes as the star(I could gaze at her for

quite some time) to Michelle Pfeiffer(I could gaze at her at full magical powers even longer) playing the horrible witch to Robert Deniro playing a nancy-boy air pirate to perfection. Charlie Cox as the lead Tristan is affable and credible and we get some very good work from a group of guys playing the sons out to be king who are constantly and consistently trying to kill off each other. Mark Strong, Jason Flemyng, and Ruppert Everett plays their roles well in both life and death(liked this whole thread as well). Peter O'Toole plays the dying killer daddy and watch for funny man Ricky Gervais who made me laugh more than anything in the entire film in his brief five minutes(nice feet). But the real power in the film is the novel by Neil Gaiman and the script made from his creative and fertile mind. Stardust creates its own mythology and its own world and it works."

Label: 1

Review: b'This picture seemed way to slanted, it\'s almost as bad as the drum beating of the right wing kooks who say everything is rosy in Iraq. It paints a picture so unredeemable that I can\'t help but wonder about it\'s legitimacy and bias. Also it seemed to meander from being about the murderous carnage of our troops to the lack of health care in the states for PTSD. To me the subject matter seemed confused, it only cared about portraying the military in a bad light, as A) an organization that uses mind control to turn ordinary peace loving civilians into baby killers and B) an organization that once having used and spent the bodies of it\'s soldiers then discards them to the despotic bureaucracy of the V.A. This is a legitimate argument, but felt off topic for me, almost like a movie in and of itself. I felt that "The War Tapes" and "Blood of my Brother" were much more fair and let the viewer draw some conclusions of their own rather than be beaten over the head with the film makers viewpoint. F-

Label: 0

Where

Label 0 = neg

Label 1 = pos

```
[ ]: # validation partition without preprocessing
rawVal = tf.keras.utils.text_dataset_from_directory(
    os.path.join(datasetDir, 'aclImdb/train'),
    batch_size = BATCH_SIZE,
    validation_split = PARTITION_VAL,
    subset = 'validation',
    seed = SEED
)
```

Found 25000 files belonging to 2 classes.

Using 5000 files for validation.

```
[ ]: # train partition 1 without preprocessing
rawTrain2 = tf.keras.utils.text_dataset_from_directory(
    os.path.join(datasetDir, 'aclImdb/test'),
    batch_size = BATCH_SIZE,
```



```

validation_split = PARTITION_VAL,
subset = 'training',
seed = SEED
)

```

Found 25000 files belonging to 2 classes.
Using 20000 files for training.

```

[ ]: # testing partition without preprocessing
rawTest = tf.keras.utils.text_dataset_from_directory(
    os.path.join(datasetDir, 'aclImdb/test'),
    batch_size = BATCH_SIZE,
    validation_split = PARTITION_VAL,
    subset = 'validation',
    seed = SEED
)

```

Found 25000 files belonging to 2 classes.
Using 5000 files for validation.

```

[ ]: # joining training data from the two folders
# to get the final training data
rawTrain = rawTrain1.concatenate(rawTrain2)
rawTrain

```

```

[ ]: <_ConcatenateDataset element_spec=(TensorSpec(shape=(None,), dtype=tf.string,
name=None), TensorSpec(shape=(None,), dtype=tf.int32, name=None))>

```

```

[ ]: # few samples of the training partition
for textBatch, labelBatch in rawTrain.take(1):
    for i in range(20, 30):
        print("Review: ", textBatch.numpy()[i])
        print("Label: ", labelBatch.numpy()[i])
print("Where")
print("Label 0 =", rawTrain1.class_names[0])
print("Label 1 =", rawTrain1.class_names[1])

```

Review: b'NOTHING (3+ outta 5 stars) Another weird premise from the director of the movie "Cube". This time around there are two main characters who find themselves and their home transported to a mysterious white void. There is literally NOTHING outside of their small two-story house. Intriguing to be sure, but I thought the comedic tone established for this movie from the get-go was extremely ill-conceived. There needs to be some humour, certainly... and I have no problem with the humour that was eventually derived from the plight of our two heroes (their final "showdown" was definitely a hoot)... but I really think the movie would have been a lot better off if it had stayed more rooted in reality in the beginning. After watching the movie I watched the "Making of" feature on the DVD and a short trailer at the end is almost totally devoid of

the "sillier" comedic aspects... making it look like a completely different (and slightly better) movie. The last half hour of the movie is where things really start to come together... similar in a way to the recent movie "Primer." The actors are fine when they are not overdoing the comedy shtick. They are really quite believable in their more "normal" moments. I was probably ready to write this movie off as a failed experiment at the midway point... but it won me over by the end. (And keep watching past the credits for the final scene... just don\'t ask me to explain it.)'

Label: 1

Review: b"What else can you say about this movie,except that it's plain awful.Tina Louise and Adam West are the reasons why to see this,but,that's it,but their talents are wasted in this junk.I think that they used a double in some of Adam's scenes,like when he's running because you can't see his face.If Adam was embarrassed in being in Zombie Nightmare,just think what he must've felt about appearing in this??? If it was before or after,I'm not sure,but,still,Zombie Nightmare is a classic(check out the Mystery Science Theater 3000 version first and last)compared to this.The gang is very annoying and over-acting by some of the actors.A rip-off of The Wild One starring Marlon Brando,of course.Tina looks stunning though.I hope her and Adam got a good paycheck!! Pass!"

Label: 0

Review: b'I could not believe how awful this film was; I rarely watch commercial TV, but thought "Well, Diane Keaton is always worth watching". I stand corrected. Everyone involved should be hanging their heads in shame.

I realize there are not a lot of great roles for women of a certain age, but the script to this was so inept, clich\xc3\xa9 and baffling that I am surprised it ever got into development or that Ms. Keaton thought she could make a silk purse out of this sow\'s ear. None of the characters had a shred of believability and were so incredibly unlikeable. The acting looked like exercises in a BEGINNING class - I stared in open-mouthed horror through most of this wondering "What were they thinking?". Very, very sad that it has come to this. Don\'t waste your time.'

Label: 0

Review: b'The performances in this movie were fantastic. The dialogue was great. Jason Patric delivered a fantastic performance as "Kid" Collins in this wonderful adaptation of the Jim Thompson novel. Far superior to "The Grifters", which was a good movie, this film really stayed true to the pulp fiction/film noir roots from which the story came. I recommend this movie to all film noir fans.'

Label: 1

Review: b'I'm a Jean Harlow fan, because she had star quality. I don't think her movies are good and I don't even think that she was a good actress, but she certainly was Great in comedies. Every bit of comedy in The Girl from Missouri is very good. But this movie is perhaps more like a love story. Jean Harlow is wonderful in this one and you can forget the rest of the cast - their performances bring nothing new. It always impresses me much to think that Harlow's beautiful body was that of an ill woman. Well, in this movie she does look beautiful."

Label: 0

Review: b"I'm probably one of the biggest Nancy Drew fans out there. I've read every book three times over and I've played a lot of the Nancy drew games. I Loved this movie. It kept you entertained the whole time you watched it. I went with about 10 of my friends and everyone LOVED it. There were three woman sitting behind us who appeared to be in their late 30's to early 40's and I asked them how they liked it, they said they loved it! So you see it will be an entertainment to all ages. You just have to give it a chance. And it teaches a lesson too, just be yourself even if everyone around you is exactly alike. So overall, this move was great. I'm going to see it a second time now! So stop bashing it please. Its a really good movie!"

Label: 1

Review: b"When I heard the plot for this movie I simply had to see it, I mean whole cities being wiped out by killer tomatoes! Sadly the title is about as funny as it gets.

Led by Detective Dick Mason, a special team of military and scientists (including Greg Colburn who never takes his SCUBA outfit off and Lt. Finletter who is never pictured without his parachute trailing behind) 'Attack Of The Killer Tomatoes' is a parody of B-Movies, in particular Japanese horror of the 1950's. The film begins with a standard sized tomato being discovered by a women washing up in her kitchen before we find ourselves in a middle of a crime scene as the tomato has supposedly murdered this lady, and let me tell you it doesn't get any saner as the film progresses! To be fair there are a couple of funny moments, for instance anytime the Japanese scientist Dr Nokitofa speaks his voice is dubbed over in an American accent, or when disguise expert Sam Smith infiltrates the tomatoes 'hey, can somebody please pass the ketchup?'. Equally this film was probably a lot funnier in 1978 with the whole so bad its good concept. Unfortunately for 'Attack Of The Killer Tomatoes' spoof films such as the 'Airplane' and 'Naked Gun' series have been released and done this kind of comedy a lot better since.

The acting is atrocious; there is zero continuity in the editing and it just feels genuinely slow and lacking energy. For a parody film to work you need a lot of things happening at once, one gag after the over. The singing in the film seems pointless and the adverts for the furniture store that flash across the screen are damn right bizarre, even for this film. Ultimately, however, you can see why this film is a cult one; I can't see many people being indifferent to it. Unfortunately terrible would be the way I would sum this up."

Label: 0

Review: b"I am shocked. Shocked and dismayed that the 428 of you IMDB users who voted before me have not given this film a rating of higher than 7. 7?!?? - that's a C!. If I could give FOBH a 20, I'd gladly do it. This film ranks high atop the pantheon of modern comedy, alongside Half Baked and Mallrats, as one of the most hilarious films of all time. If you know _anything_ about rap music - YOU MUST SEE THIS!! If you know nothing about rap music - learn something!, and then see this! Comparisons to 'Spinal Tap' fail to appreciate the inspired genius of this unique film. If you liked Bob Roberts, you'll love this. Watch it and vote it a 10!"

Label: 1

Review: b"I agree with another user here and have to say that this is one of

the best Kung Fu movies ever! I watched this as a kid and absolutely loved it! The scaffolding scenes are brilliant and you can really empathise with this guy because he is treated as an outcast. Nice humour and fantastic kung fu this movie rocks! If you like Kung Fu you would love this!!!'

Label: 1

Review: b'The Western society has been fed ideas about India being a poor country. Movies like these only make those beliefs stronger. Such illustrations make it all the more difficult for Indians to be accepted abroad. Agreed there are poor and homeless in India, but why is there no representation of educated people if not the successful ones.

I totally hated the idea of the movie portraying Patrick Swayze as another Mother Teresa. In my opinion this movie has shown India in a very bad light giving wrong notions. It is unjust to discuss only one aspect of the society. Exactly the reason why people ask me, "When we go to India, can we hire an elephant right outside the airport so we do not have to walk on the roads so full of filth and snakes?"

Those who want a second opinion on contemporary Indian society should watch "Monsoon Wedding".'

Label: 0

Where

Label 0 = neg

Label 1 = pos

6 Data Preprocessing

```
[ ]: # removes the HTML tags and punctuation in the given text
def stripHTMLAndPunctuation(inputText):
    noHTML = tf.strings.regex_replace(tf.strings.lower(inputText),
                                      "(<br />|(</?p>)", ' ')
    return tf.strings.regex_replace(noHTML,
                                    ' [%s]' % re.escape(string.punctuation),
                                    '')
```

```
[ ]: # maximum vocab size
MAX_FEATURES = 10000

# length to text vector
SEQ_LEN = 250
```

```
[ ]: # layer to create vectors from raw text
vectorizeLayer = layers.TextVectorization(
    standardize = stripHTMLAndPunctuation,
    max_tokens = MAX_FEATURES,
    output_mode = 'int',
    output_sequence_length = SEQ_LEN)
```

```
[ ]: # adapting the layer to learn the vocab
vectorizeLayer.adapt(rawTrain.map(lambda x, y: x))
```

```
[ ]: pickle.dump({'config': vectorizeLayer.get_config(),
                 'weights': vectorizeLayer.get_weights()}
               , open("/content/tv_layer.pkl", "wb"))
```

```
[ ]: # exploring the vocab
print(vectorizeLayer.get_vocabulary()[100: 200])
print('Vocabulary size: {}'.format(len(vectorizeLayer.get_vocabulary())))
```

```
[np.str_('characters'), np.str_('after'), np.str_('think'), np.str_('watch'),
np.str_('many'), np.str_('seen'), np.str_('two'), np.str_('character'),
np.str_('being'), np.str_('never'), np.str_('love'), np.str_('acting'),
np.str_('plot'), np.str_('did'), np.str_('where'), np.str_('best'),
np.str_('know'), np.str_('little'), np.str_('show'), np.str_('life'),
np.str_('ever'), np.str_('does'), np.str_('your'), np.str_('better'),
np.str_('end'), np.str_('off'), np.str_('over'), np.str_('still'),
np.str_('these'), np.str_('say'), np.str_('scene'), np.str_('man'),
np.str_('here'), np.str_('why'), np.str_('while'), np.str_('scenes'),
np.str_('such'), np.str_('something'), np.str_('go'), np.str_('through'),
np.str_('should'), np.str_('im'), np.str_('those'), np.str_('back'),
np.str_('real'), np.str_('watching'), np.str_('now'), np.str_('thing'),
np.str_('didnt'), np.str_('doesnt'), np.str_('actors'), np.str_('years'),
np.str_('though'), np.str_('another'), np.str_('before'), np.str_('funny'),
np.str_('actually'), np.str_('nothing'), np.str_('makes'), np.str_('find'),
np.str_('work'), np.str_('look'), np.str_('same'), np.str_('lot'),
np.str_('going'), np.str_('every'), np.str_('new'), np.str_('few'),
np.str_('old'), np.str_('again'), np.str_('us'), np.str_('part'),
np.str_('director'), np.str_('cant'), np.str_('things'), np.str_('cast'),
np.str_('thats'), np.str_('quite'), np.str_('want'), np.str_('pretty'),
np.str_('around'), np.str_('seems'), np.str_('world'), np.str_('down'),
np.str_('young'), np.str_('got'), np.str_('enough'), np.str_('fact'),
np.str_('take'), np.str_('however'), np.str_('horror'), np.str_('give'),
np.str_('both'), np.str_('big'), np.str_('thought'), np.str_('ive'),
np.str_('may'), np.str_('between'), np.str_('own'), np.str_('long')]
Vocabulary size: 10000
```

```
[ ]:
```

```
[ ]: # vectorizes the text
def vectorizeText(inputText, label):
    return vectorizeLayer(tf.expand_dims(inputText, -1)), label
```

```
[ ]: # explore effect of vectorization on a review
textBatch, labelBatch = next(iter(rawTrain))
print("Review:", textBatch[BATCH_SIZE // 2])
```


7 Caching for Speed

```
[ ]: AUTOTUNE = tf.data.AUTOTUNE
```

```
[ ]: trainData = trainData.cache().prefetch(buffer_size = AUTOTUNE)
     valData = valData.cache().prefetch(buffer_size = AUTOTUNE)
     testData = testData.cache().prefetch(buffer_size = AUTOTUNE)
```

8 LSTM Model

```
[ ]: # embedding layer output size
     EMBEDDING_DIM = 64
```

```
[ ]: # model definition
     modelLSTM = tf.keras.Sequential([
         tf.keras.layers.Embedding(
             input_dim = MAX_FEATURES + 1,
             output_dim = EMBEDDING_DIM,
             mask_zero=True),
         #tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(200)),
         tf.keras.layers.LSTM(200),
         #tf.keras.layers.Dropout(0.2),
         tf.keras.layers.Dense(64, activation = "relu"),
         tf.keras.layers.Dense(1)
     ])
```

```
[ ]: modelLSTM.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)
dense_1 (Dense)	?	0 (unbuilt)

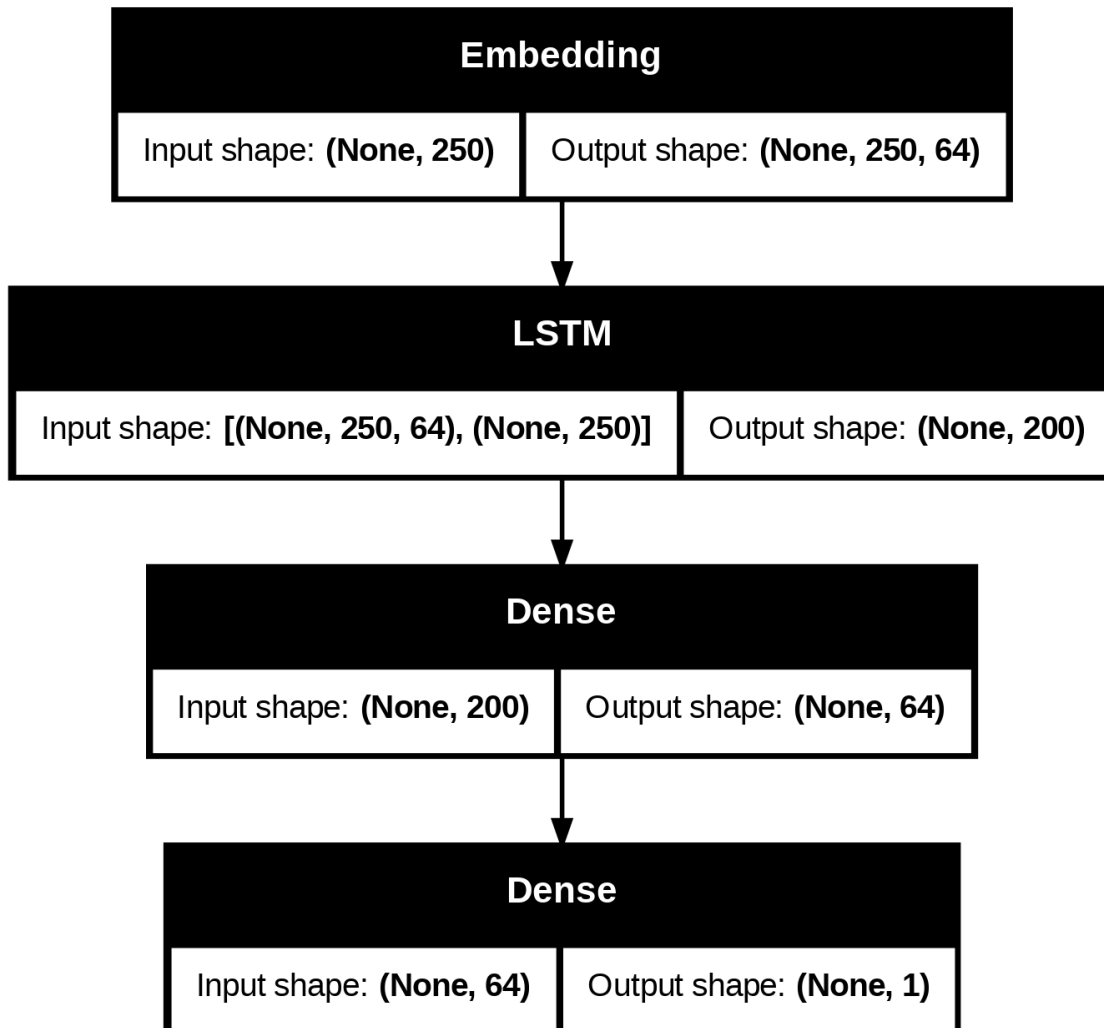
Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

```
[ ]: from tensorflow.keras.utils import plot_model
modelLSTM.build(input_shape=(None, SEQ_LEN))
plot_model(modelLSTM, show_shapes = True)
```

[]:



```
[ ]: modelLSTM.compile(loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),
optimizer=tf.keras.optimizers.Adam(1e-4),
metrics=['accuracy'])
```

```
[ ]: EPOCHS_LSTM = 5
```



```
[ ]: # model training
history = modelLSTM.fit(
    trainData,
    validation_data = valData,
    epochs = EPOCHS_LSTM)

Epoch 1/5
314/314          15s 32ms/step -
accuracy: 0.5609 - loss: 0.6480 - val_accuracy: 0.8224 - val_loss: 0.3996
Epoch 2/5
314/314          8s 25ms/step -
accuracy: 0.8564 - loss: 0.3431 - val_accuracy: 0.8722 - val_loss: 0.3255
Epoch 3/5
314/314          8s 26ms/step -
accuracy: 0.8972 - loss: 0.2619 - val_accuracy: 0.8730 - val_loss: 0.3013
Epoch 4/5
314/314          8s 27ms/step -
accuracy: 0.9127 - loss: 0.2353 - val_accuracy: 0.8780 - val_loss: 0.3205
Epoch 5/5
314/314          8s 25ms/step -
accuracy: 0.9222 - loss: 0.2134 - val_accuracy: 0.8714 - val_loss: 0.4123
```

```
[ ]: # evaluation on test dataset
testLoss, testAccuracy = modelLSTM.evaluate(testData)
!mkdir -p modelLSTM
modelLSTM.save('/content/modelLSTM/modelLSTM.keras')

print("Loss: ", testLoss)
print("Accuracy: ", testAccuracy)
```

```
40/40          1s 13ms/step -
accuracy: 0.8783 - loss: 0.3923
Loss: 0.38509806990623474
Accuracy: 0.8755999803543091
```

```
[ ]: # plotting training loss and accuracy
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(1, EPOCHS_LSTM + 1)

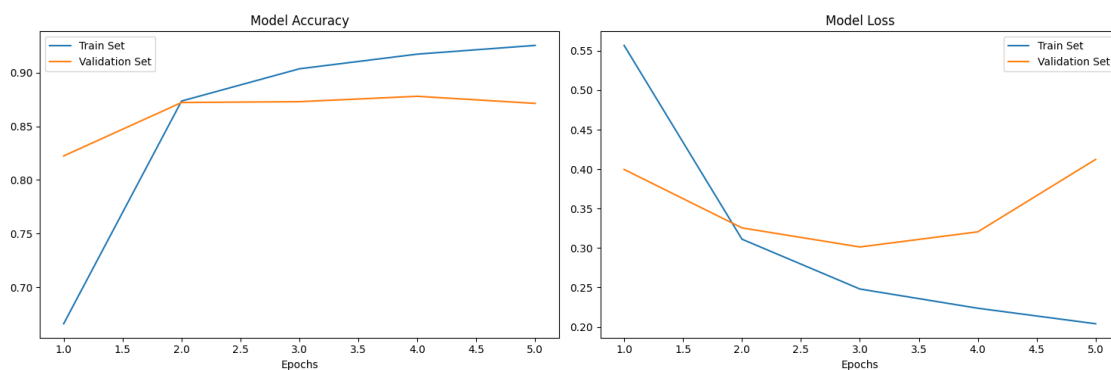
plt.figure(figsize=(15,5))

plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Train Set')
plt.plot(epochs_range, val_acc, label='Validation Set')
```

```
plt.legend(loc="best")
plt.xlabel('Epochs')
plt.title('Model Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Train Set')
plt.plot(epochs_range, val_loss, label='Validation Set')
plt.legend(loc="best")
plt.xlabel('Epochs')
plt.title('Model Loss')

plt.tight_layout()
plt.show()
```



```
[ ]: yPred = np.array(modelLSTM.predict(testData) >= 0)
```

40/40 1s 12ms/step

```
[ ]: yPred
```

```
[ ]: array([[ True],
           [ True],
           [ True],
           ...,
           [ True],
           [False],
           [ True]])
```

```
[ ]: testData
```

```
[ ]: <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 250), dtype=tf.int64,
name=None), TensorSpec(shape=(None,), dtype=tf.int32, name=None))>
```

```
[ ]: yActual = np.concatenate([y for x, y in testData], axis=0)
```

```
[ ]: yActual.shape
```

```
[ ]: (5000,)
```

```
[ ]: yPredNumeric = np.reshape(1 * yPred, (yPred.shape[0],))  
print(yPredNumeric, yPredNumeric.shape, sep = "\n")
```

```
[1 1 1 ... 1 0 1]  
(5000,)
```

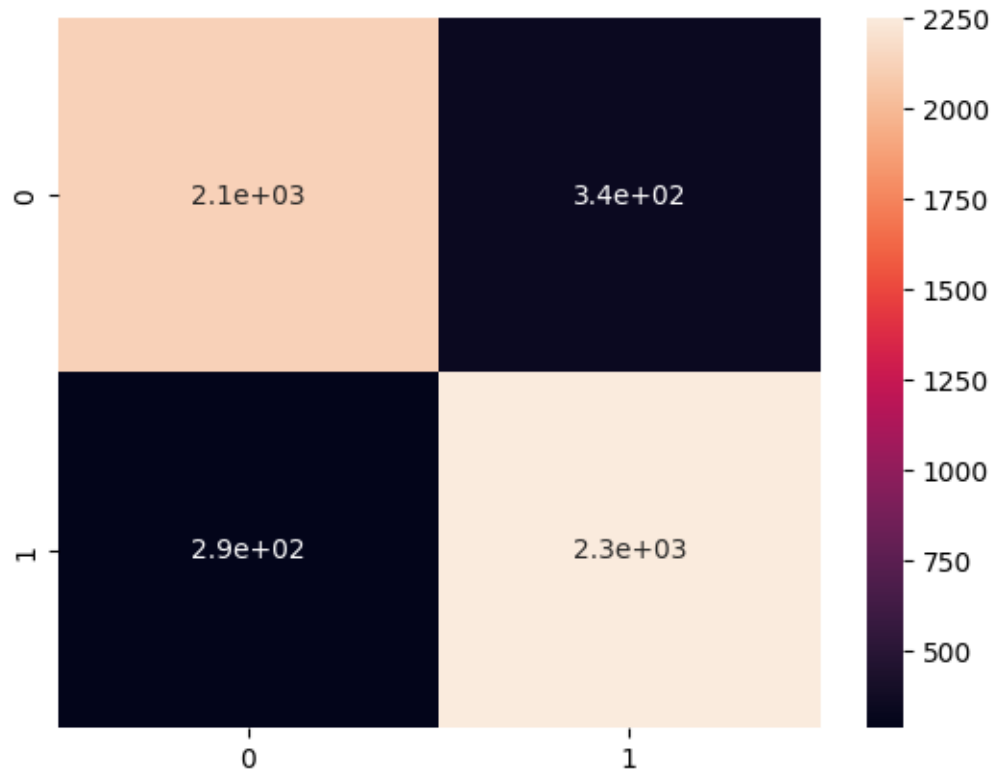
```
[ ]: # model evaluation  
print("accuracy of LSTM based model = {}".format(accuracy_score(yActual, yPredNumeric)))  
print(classification_report(yActual, yPredNumeric))
```

accuracy of LSTM based model = 0.874

	precision	recall	f1-score	support
0	0.88	0.86	0.87	2462
1	0.87	0.89	0.88	2538
accuracy			0.87	5000
macro avg	0.87	0.87	0.87	5000
weighted avg	0.87	0.87	0.87	5000

```
[ ]: # confusion matrix  
cfm = confusion_matrix(yActual, yPredNumeric)  
sns.heatmap(cfm, annot = True)
```

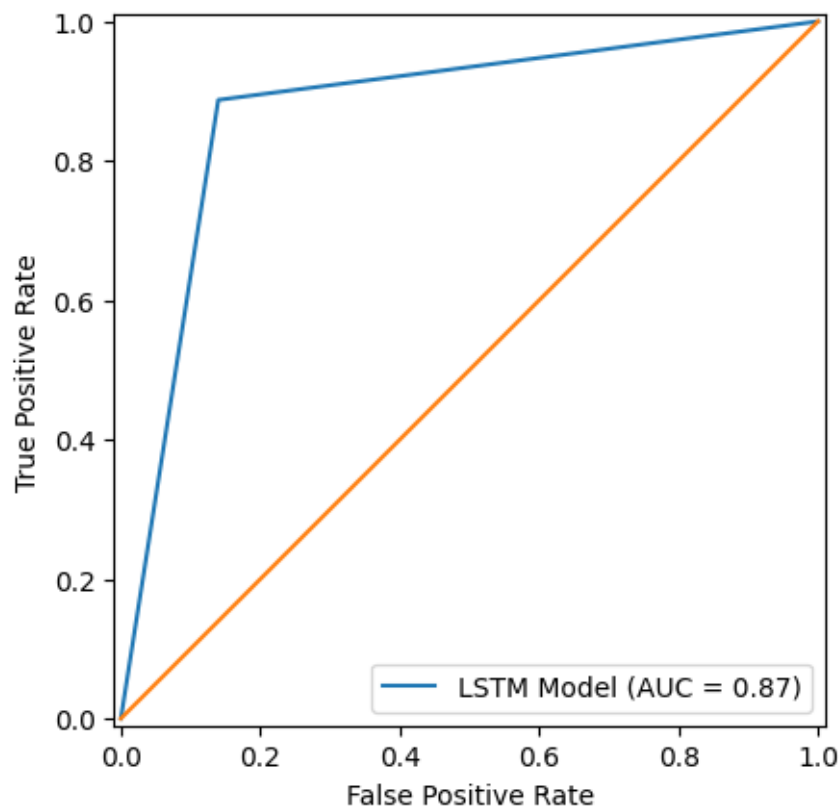
```
[ ]: <Axes: >
```



```
[ ]: tn, fp, fn, tp = confusion_matrix(yActual, yPredNumeric).ravel()
      print("specificity =", tn / (tn + fp))
```

specificity = 0.8602761982128351

```
[ ]: # ROC curve
      fpr, tpr, thresholds = roc_curve(yActual, yPredNumeric)
      roc_auc = auc(fpr, tpr)
      display = RocCurveDisplay(fpr=fpr, tpr=tpr, roc_auc=roc_auc,
                                estimator_name='LSTM Model')
      display.plot()
      plt.plot(fpr, fpr)
      plt.show()
```



9 CNN Based Model

```
[ ]: LAMBDA = 0.0001
```

```
[ ]: # model definition
modelCNN = tf.keras.Sequential([
    tf.keras.layers.Embedding(
        input_dim = MAX_FEATURES + 1,
        output_dim = EMBEDDING_DIM,
        mask_zero=True),
    tf.keras.layers.Conv1D(filters = 64,
                           kernel_regularizer=regularizers.l2(LAMBDA),
                           kernel_initializer='he_uniform',
                           kernel_size = 3,
                           activation = "relu"),
    tf.keras.layers.MaxPooling1D(2),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Conv1D(filters = 32, kernel_size = 2,
                           kernel_regularizer=regularizers.l2(LAMBDA),
                           kernel_initializer='he_uniform',
```

```

        activation = "relu"),
tf.keras.layers.GlobalAveragePooling1D(),
tf.keras.layers.Dropout(0.2),
tf.keras.layers.Dense(64, kernel_regularizer=regularizers.l2(LAMBDA),
        kernel_initializer='he_uniform',
        activation = "relu"),
tf.keras.layers.Dense(1)
])

```

```
[ ]: modelCNN.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
conv1d (Conv1D)	?	0 (unbuilt)
max_pooling1d (MaxPooling1D)	?	0
dropout (Dropout)	?	0
conv1d_1 (Conv1D)	?	0 (unbuilt)
global_average_pooling1d (GlobalAveragePooling1D)	?	0
dropout_1 (Dropout)	?	0
dense_2 (Dense)	?	0 (unbuilt)
dense_3 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

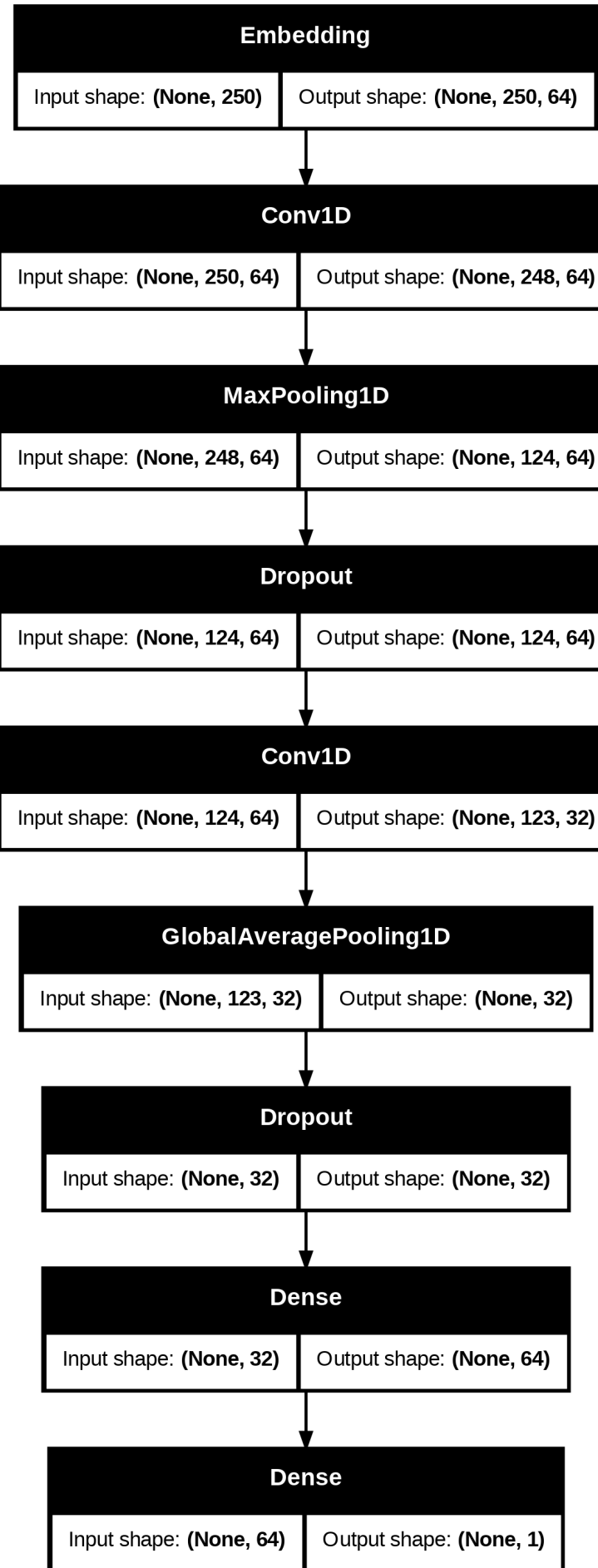
Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

```
[ ]: from tensorflow.keras.utils import plot_model
modelCNN.build(input_shape=(None, SEQ_LEN))
```

```
plot_model(modelCNN, show_shapes = True)
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/layer.py:965:  
UserWarning: Layer 'conv1d' (of type Conv1D) was passed an input with a mask  
attached to it. However, this layer does not support masking and will therefore  
destroy the mask information. Downstream layers will not see the mask.  
warnings.warn(  
[ ]:
```




```
[ ]: # evaluation on test dataset
testLoss, testAccuracy = modelCNN.evaluate(testData)
!mkdir -p modelCNN
modelCNN.save('/content/modelCNN/modelCNN.keras')

print("Loss: ", testLoss)
print("Accuracy: ", testAccuracy)
```

```
40/40          0s 3ms/step -
accuracy: 0.8803 - loss: 0.3038
Loss: 0.3052511215209961
Accuracy: 0.8822000026702881
```

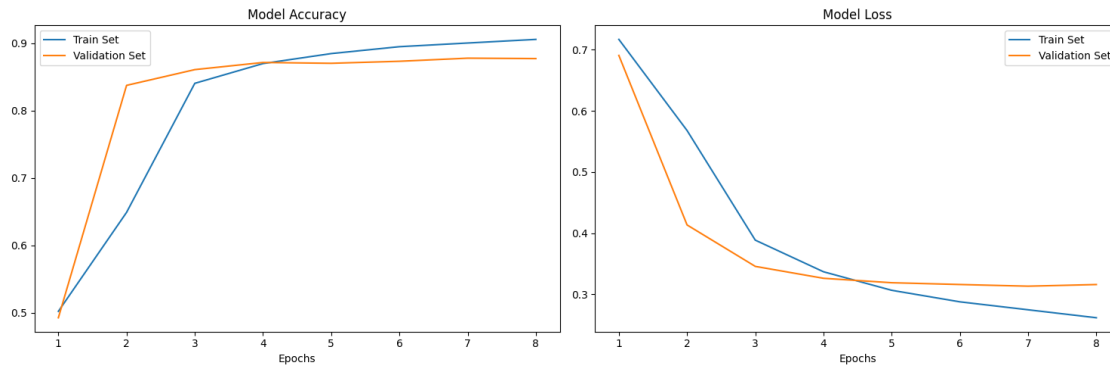
```
[ ]: # plotting training loss and accuracy
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(1, EPOCHS_CNN + 1)

plt.figure(figsize=(15,5))

plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Train Set')
plt.plot(epochs_range, val_acc, label='Validation Set')
plt.legend(loc="best")
plt.xlabel('Epochs')
plt.title('Model Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Train Set')
plt.plot(epochs_range, val_loss, label='Validation Set')
plt.legend(loc="best")
plt.xlabel('Epochs')
plt.title('Model Loss')

plt.tight_layout()
plt.show()
```



```
[ ]: yPred = np.array(modelCNN.predict(testData) >= 0)
```

```
40/40          1s 11ms/step
```

```
[ ]: yActual = np.concatenate([y for x, y in testData], axis=0)
```

```
[ ]: yPredNumeric = np.reshape(1 * yPred, (yPred.shape[0],))
print(yPredNumeric, yPredNumeric.shape, sep = "\n")
```

```
[1 1 1 ... 1 0 1]
(5000,)
```

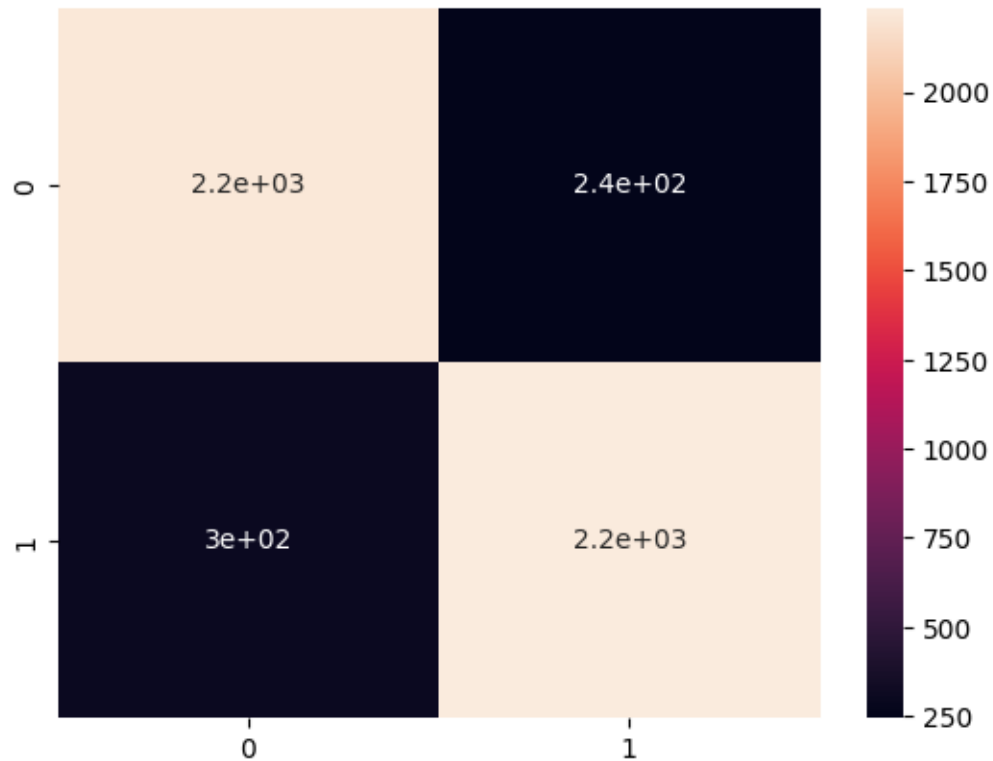
```
[ ]: # model evaluation
print("accuracy of CNN based model = {}".format(accuracy_score(yActual,
    ↪ yPredNumeric)))
print(classification_report(yActual, yPredNumeric))
```

```
accuracy of CNN based model = 0.891
```

	precision	recall	f1-score	support
0	0.88	0.90	0.89	2462
1	0.90	0.88	0.89	2538
accuracy			0.89	5000
macro avg	0.89	0.89	0.89	5000
weighted avg	0.89	0.89	0.89	5000

```
[ ]: # confusion matrix
cfm = confusion_matrix(yActual, yPredNumeric)
sns.heatmap(cfm, annot = True)
```

```
[ ]: <Axes: >
```

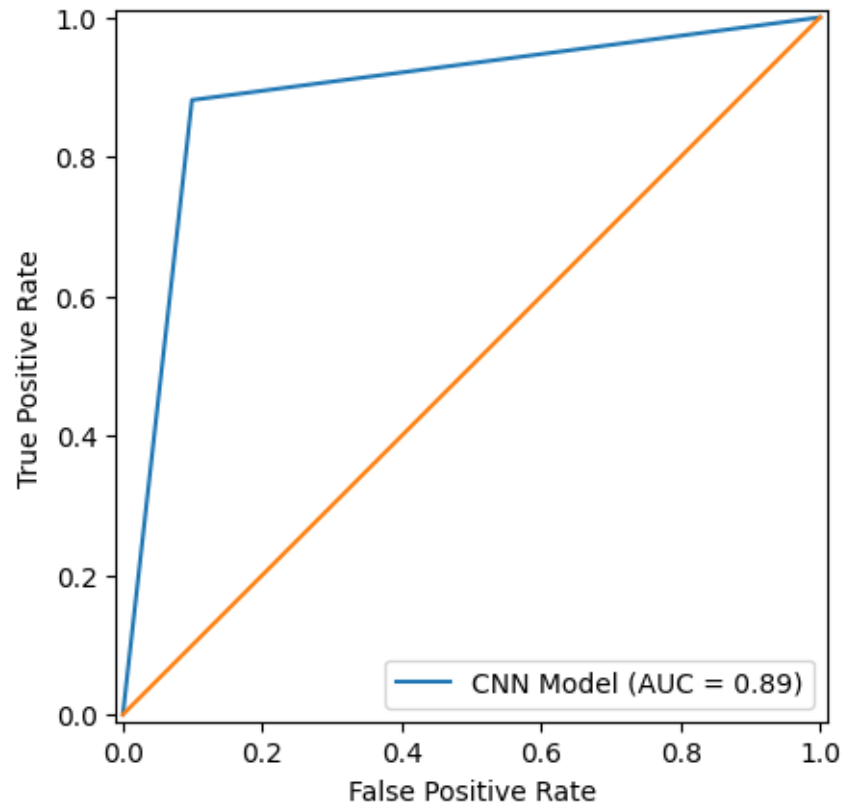


```
[ ]: tn, fp, fn, tp = confusion_matrix(yActual, yPredNumeric).ravel()
print("specificity =", tn / (tn + fp))
```

specificity = 0.90089358245329

```
[ ]: # ROC curve
fpr, tpr, thresholds = roc_curve(yActual, yPredNumeric)
roc_auc = auc(fpr, tpr)
display = RocCurveDisplay(fpr=fpr, tpr=tpr, roc_auc=roc_auc,
                           estimator_name='CNN Model')

display.plot()
plt.plot(fpr, fpr)
plt.show()
```



10 LSTM-CNN Based Model

```
[ ]: # model definition
modelCombine = tf.keras.Sequential([
    tf.keras.layers.Embedding(
        input_dim = MAX_FEATURES + 1,
        output_dim = EMBEDDING_DIM,
        mask_zero=True),
    tf.keras.layers.LSTM(200,
        kernel_regularizer=regularizers.l2(LAMBDA),
        return_sequences = True),
    tf.keras.layers.Conv1D(filters = 64, kernel_size = 3,
        kernel_regularizer=regularizers.l2(LAMBDA),
        kernel_initializer='he_uniform',
        activation = "relu"),
    tf.keras.layers.MaxPooling1D(2),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Conv1D(filters = 32, kernel_size = 2,
        kernel_regularizer=regularizers.l2(LAMBDA),
        kernel_initializer='he_uniform',
```

```

        activation = "relu"),
tf.keras.layers.GlobalAveragePooling1D(),
tf.keras.layers.Dropout(0.2),
tf.keras.layers.Dense(64,
                        kernel_regularizer=regularizers.l2(LAMBDA),
                        kernel_initializer='he_uniform',
                        activation = "relu"),
tf.keras.layers.Dense(1)
])

```

```
[ ]: modelCombine.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	0 (unbuilt)
lstm_1 (LSTM)	?	0 (unbuilt)
conv1d_2 (Conv1D)	?	0 (unbuilt)
max_pooling1d_1 (MaxPooling1D)	?	0
dropout_2 (Dropout)	?	0
conv1d_3 (Conv1D)	?	0 (unbuilt)
global_average_pooling1d_1 (GlobalAveragePooling1D)	?	0
dropout_3 (Dropout)	?	0
dense_4 (Dense)	?	0 (unbuilt)
dense_5 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

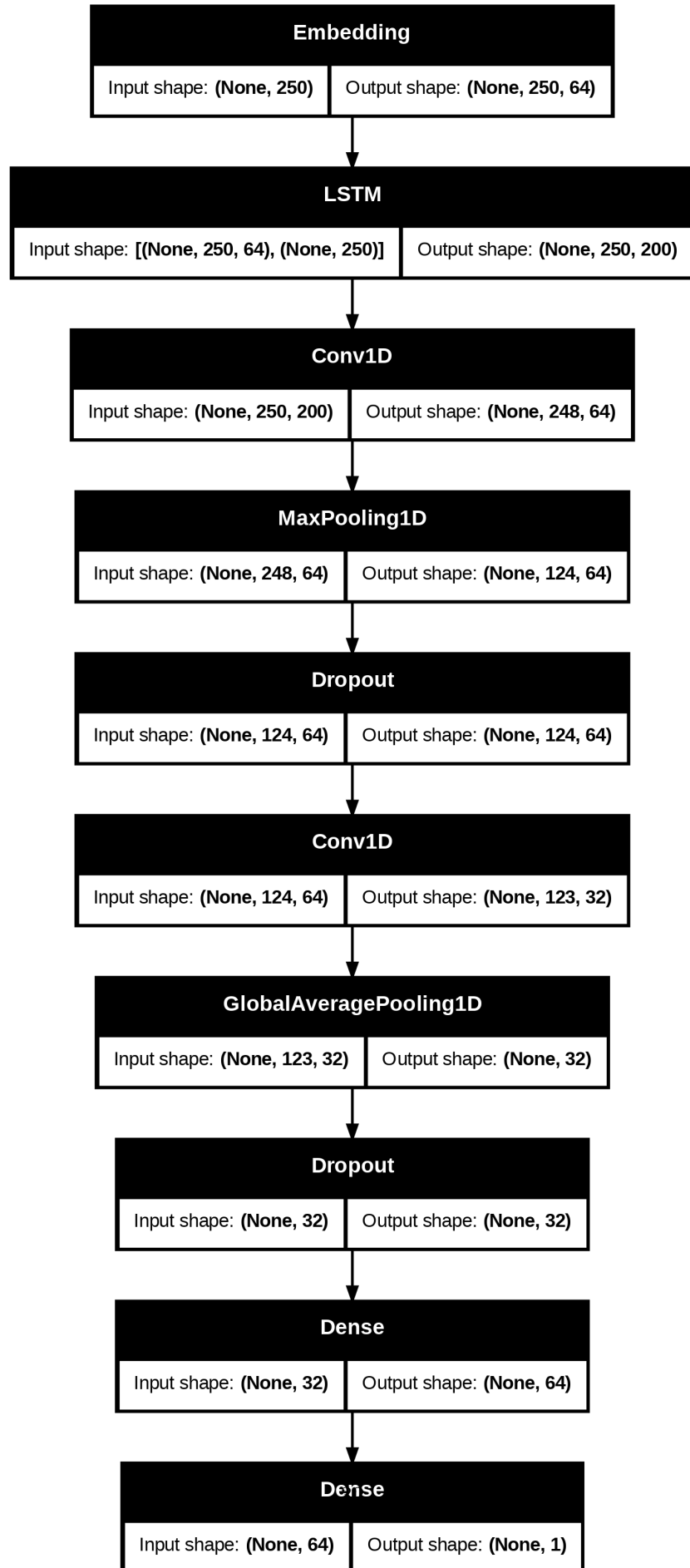
Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

```
[ ]: from tensorflow.keras.utils import plot_model
      modelCombine.build(input_shape=(None, SEQ_LEN))
      plot_model(modelCombine, show_shapes = True)
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/layer.py:965:
UserWarning: Layer 'conv1d_2' (of type Conv1D) was passed an input with a mask
attached to it. However, this layer does not support masking and will therefore
destroy the mask information. Downstream layers will not see the mask.
  warnings.warn(
```

```
[ ]:
```




```
[ ]: modelCombine.compile(loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),
                           optimizer=tf.keras.optimizers.Adam(1e-4),
                           metrics=['accuracy'])
```

```
[ ]: EPOCHS_Combine = 6
```

```
[ ]: # model training
history = modelCombine.fit(
    trainData,
    validation_data = valData,
    epochs = EPOCHS_Combine)
```

Epoch 1/6

/usr/local/lib/python3.11/dist-packages/keras/src/layers/layer.py:965:
UserWarning: Layer 'conv1d_2' (of type Conv1D) was passed an input with a mask
attached to it. However, this layer does not support masking and will therefore
destroy the mask information. Downstream layers will not see the mask.
warnings.warn(

314/314 14s 34ms/step -
accuracy: 0.5371 - loss: 0.7045 - val_accuracy: 0.7758 - val_loss: 0.5162

Epoch 2/6

314/314 19s 33ms/step -
accuracy: 0.8181 - loss: 0.4302 - val_accuracy: 0.8464 - val_loss: 0.3884

Epoch 3/6

314/314 21s 33ms/step -
accuracy: 0.8789 - loss: 0.3307 - val_accuracy: 0.8514 - val_loss: 0.3853

Epoch 4/6

314/314 20s 32ms/step -
accuracy: 0.8951 - loss: 0.2954 - val_accuracy: 0.8614 - val_loss: 0.3767

Epoch 5/6

314/314 10s 33ms/step -
accuracy: 0.9061 - loss: 0.2665 - val_accuracy: 0.8612 - val_loss: 0.4295

Epoch 6/6

314/314 21s 33ms/step -
accuracy: 0.9116 - loss: 0.2523 - val_accuracy: 0.8612 - val_loss: 0.5455

```
[ ]: # evaluation on test dataset
testLoss, testAccuracy = modelCombine.evaluate(testData)
!mkdir -p modelCombine
modelCombine.save('/content/modelCombine/modelCombine.keras')

print("Loss: ", testLoss)
print("Accuracy: ", testAccuracy)
```

40/40 1s 13ms/step -
accuracy: 0.8674 - loss: 0.4933
Loss: 0.5092151165008545
Accuracy: 0.8611999750137329

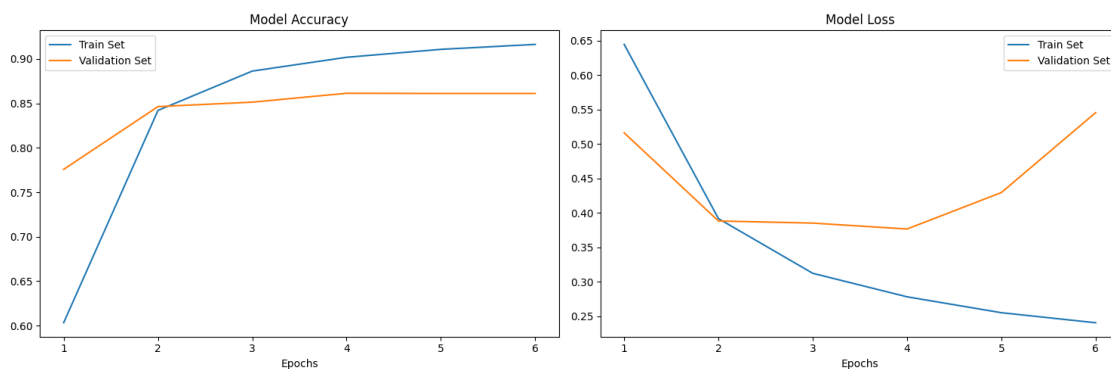
```
[ ]: # plotting training loss and accuracy
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(1, EPOCHS_Combine + 1)

plt.figure(figsize=(15,5))
#plt.axis([0, 7, 0, 1])

plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Train Set')
plt.plot(epochs_range, val_acc, label='Validation Set')
plt.legend(loc="best")
plt.xlabel('Epochs')
plt.title('Model Accuracy')
#plt.axis([0, 7, 0.5, 1])

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Train Set')
plt.plot(epochs_range, val_loss, label='Validation Set')
plt.legend(loc="best")
plt.xlabel('Epochs')
plt.title('Model Loss')

plt.tight_layout()
#plt.axis([0, 7, 0, 1])
plt.show()
```



```
[ ]: yPred = np.array(modelCombine.predict(testData) >= 0)
```

40/40 1s 14ms/step

```
[ ]: yActual = np.concatenate([y for x, y in testData], axis=0)
```

```
[ ]: yPredNumeric = np.reshape(1 * yPred, (yPred.shape[0],))
      print(yPredNumeric, yPredNumeric.shape, sep = "\n")
```

[1 1 1 ... 1 0 1]
(5000,)

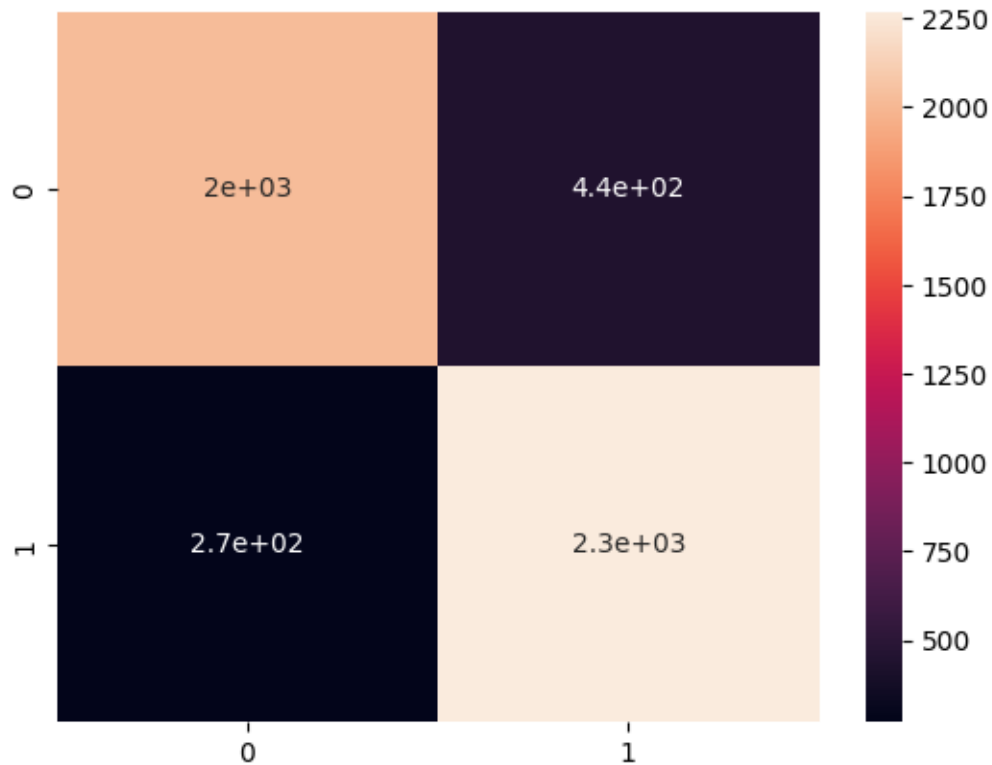
```
[ ]: # model evaluation
      print("accuracy of LSTM-CNN based model = {}".format(accuracy_score(yActual, yPredNumeric)))
      print(classification_report(yActual, yPredNumeric))
```

accuracy of LSTM-CNN based model = 0.8588

	precision	recall	f1-score	support
0	0.88	0.82	0.85	2462
1	0.84	0.89	0.87	2538
accuracy			0.86	5000
macro avg	0.86	0.86	0.86	5000
weighted avg	0.86	0.86	0.86	5000

```
[ ]: # confusion matrix
      cfm = confusion_matrix(yActual, yPredNumeric)
      sns.heatmap(cfm, annot = True)
```

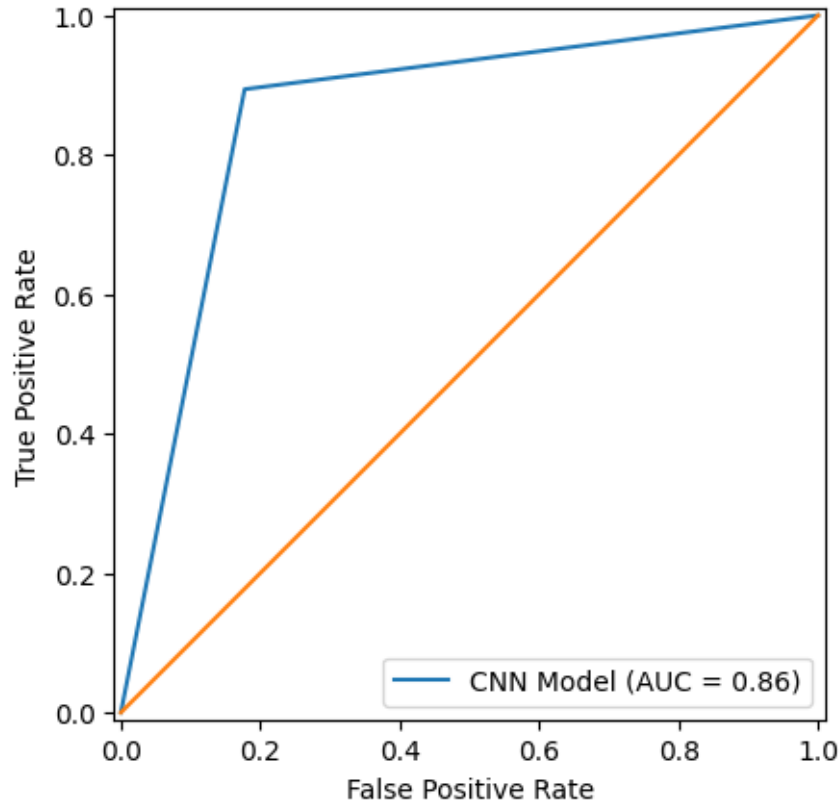
```
[ ]: <Axes: >
```



```
[ ]: tn, fp, fn, tp = confusion_matrix(yActual, yPredNumeric).ravel()
      print("specificity =", tn / (tn + fp))
```

```
specificity = 0.822502030869212
```

```
[ ]: # ROC curve
      fpr, tpr, thresholds = roc_curve(yActual, yPredNumeric)
      roc_auc = auc(fpr, tpr)
      display = RocCurveDisplay(fpr=fpr, tpr=tpr, roc_auc=roc_auc,
                                estimator_name='CNN Model')
      display.plot()
      plt.plot(fpr, fpr)
      plt.show()
```



11 Final Note on Implemented Models

In Part B, we implemented three models: - LSTM (based on Paper 3 by Qaisar, 2020)

- CNN (based on Paper 1 by Poria et al., 2016)

- Hybrid LSTM-CNN (inspired by combined approaches)

This was done to provide **comparative validation** across architectures.

While the assignment requires implementation of **one chosen paper**, we extended the work to also include CNN and Hybrid LSTM-CNN for a deeper evaluation.

12 Best Model Conclusion:

Based on our results:

- **CNN** achieved the best balance between accuracy (89%) and F1-score (0.89), showing strong generalization.

- **LSTM** achieved higher recall (0.92), making it better at reducing false negatives.

- **Hybrid LSTM-CNN** had higher training accuracy but slightly lower test generalization (86%).

Therefore, **CNN is identified as the best-performing model overall**, but the additional implementations demonstrate robustness of findings and strengthen the submission.